

Let's try to make sense of this output. First of all, we see some information about the system the test is running on ❶. I'm testing this on a macOS system, so you may see some different output here. Most importantly, we can see which versions of Python, pytest, and other packages are being used to run the test.

Next, we see the directory where the test is being run from ❷: in my case, `python_work/chapter_11`. We can see that pytest found one test to run ❸, and we can see the test file that's being run ❹. The single dot after the name of the file tells us that a single test passed, and the 100% makes it clear that all of the tests have been run. A large project can have hundreds or thousands of tests, and the dots and percentage-complete indicator can be helpful in monitoring the overall progress of the test run.

The last line tells us that one test passed, and it took less than 0.01 seconds to run the test.

This output indicates that the function `get_formatted_name()` will always work for names that have a first and last name, unless we modify the function. When we modify `get_formatted_name()`, we can run this test again. If the test passes, we know the function will still work for names like Janis Joplin.

NOTE

If you're not sure how to navigate to the right location in the terminal, see "Running Python Programs from a Terminal" on page 11. Also, if you see a message that the `pytest` command was not found, use the command `python -m pytest` instead.

A Failing Test

What does a failing test look like? Let's modify `get_formatted_name()` so it can handle middle names, but let's do so in a way that breaks the function for names with just a first and last name, like Janis Joplin.

Here's a new version of `get_formatted_name()` that requires a middle name argument:

```
name def get_formatted_name(first, middle, last):
_function.py """Generate a neatly formatted full name."""
            full_name = f"{first} {middle} {last}"
            return full_name.title()
```

This version should work for people with middle names, but when we test it, we see that we've broken the function for people with just a first and last name.

This time, running **pytest** gives the following output:

```
$ pytest
===== test session starts =====
--snip--
❶ test_name_function.py F [100%]
❷ ===== FAILURES =====
❸ _____ test_first_last_name _____
            def test_first_last_name():
                """Do names like 'Janis Joplin' work?"""
❹ >         formatted_name = get_formatted_name('janis', 'joplin')
❺ E         TypeError: get_formatted_name() missing 1 required positional
            argument: 'last'
```
